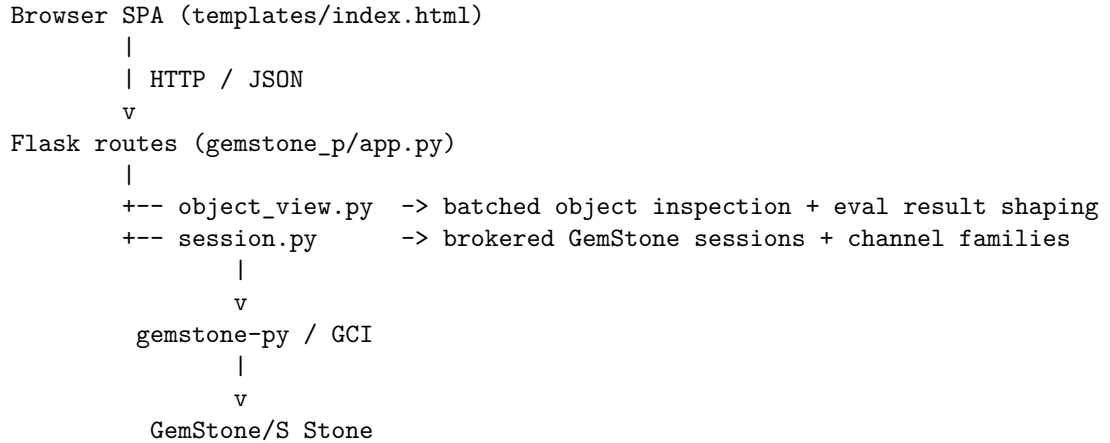


Architecture

Overview



The application is a single-page desktop-style web UI. Most user actions open windows, helper windows, or linked inspectors rather than navigating to new pages.

Frontend

`templates/index.html`

The frontend is intentionally self-contained. It provides:

- object inspectors and workspace windows
- Symbol List Browser
- Class Browser
- Codegen Explorer for selecting live classes/protocols/selectors, inspecting method source, importing/exporting selection JSON, and previewing gemstone-py generated wrappers
- helper windows for queries, hierarchy, and versions
- debugger windows
- status-log, Connection, and About/support windows, including client-side support-bundle export
- layout persistence and window-manager actions
- taskbar/window grouping and relationship arrows

Important implementation traits:

- backend-driven object tabs via `availableTabs`, `defaultTab`, and `customTabs`
- helper windows keep source-browser linkage so `Load Into Browser`, `Open In Browser`, `Raise Related`, and `Close Group` work after layout restore

- per-window browser state is cached in-memory and persisted in `localStorage`
- the SPA uses lightweight fetch helpers rather than a framework/runtime bundle

Flask Layer

`gemstone_p/app.py`

`create_app()` defines the full JSON API surface. The main route groups are:

- roots, version, health, and diagnostics metadata
- connection preflight metadata and local connection probing
- object inspector, eval, code pane, and inspector helper tabs
- Class Browser read/write actions
- Codegen Explorer read-only discovery and preview/export actions
- Symbol List Browser
- debugger
- transaction and persistent-mode control

The file also carries most of the Smalltalk snippets used to ask GemStone for browser data. Those snippets are deliberately shaped to return compact, encoded strings instead of many small GCI calls.

Object View Layer

`gemstone_p/object_view.py`

`object_view()` translates a GemStone OOP into the JSON shape the UI expects.

Current responsibilities include:

- classifying frontend basetypes (`hash`, `array`, `object`, `string`, `symbol`, and so on)
- batched fetches for instance variables, dictionary entries, and indexed elements
- backend tab metadata (`availableTabs`, `defaultTab`)
- exact Class Browser handoff metadata (`classBrowserTarget`)
- custom-tab adapters such as MagLev-style `Attributes`
- eval result shaping, including debugger metadata when evaluation halts

This layer is where the port most directly mirrors the original MagLev explorer's notion of object-specific database views.

Session Broker

`gemstone_p/session.py`

The runtime no longer does naive per-request login/logout. Instead, it uses a small broker that:

- lazily creates managed `GemStoneSession` objects
- remembers persistent-mode defaults and reapplies them to new sessions
- routes requests into channel families
- still serializes actual GCI access behind a process-global lock because the underlying client state is not safely concurrent

Important constraint:

- GCI still behaves like process-global state. The broker improves isolation and session recovery, but it does **not** make a Flask process fully parallel for GemStone work.

Current isolation model:

- the SPA can send `X-GS-Channel` so separate windows use separate managed session families
- when no explicit channel is supplied, the broker falls back to route-derived channels
- broken sessions can be dropped without tearing down the whole process

The Codegen Explorer uses this same route-derived `codegen` family and honors exact per-window channels from the frontend. Its backend routes expose stable JSON for dictionaries, classes, protocols/categories, methods, and source, then use `gemstone-py` codegen only for local preview generation. The browser keeps selection editing local: imported JSON is normalized by the backend, and users can adjust Python method names, argument names, and return annotations before preview/export. The explorer does not own generation rules; `gemstone-py` remains the source of decorators, selector rules, wrapper output, `.pyi` files, and `async-wrapper` behavior.

This design is a compromise: isolate browser/debugger flows better than a single global session, while staying honest about GemStone/GCI limits.

Testing

Python tests

- `tests/test_app.py` covers route behavior and generated Smalltalk snippets
- `tests/test_object_view.py` covers object-view shaping and eval-edge cases
- `tests/test_session.py` covers broker channel behavior

Browser tests

- `tests/ui/mock_server.py` provides a deterministic mock backend
- `tests/ui/smoke.spec.js` covers the SPA against that mock backend
- `tests/ui/live.spec.js` runs an opt-in live GemStone suite
- `npm run test:ui` runs the deterministic mock suite only
- `npm run test:ui:live` runs the live suite explicitly
- `npm run test:ui:all` runs the mock suite first and chains the live suite automatically when the required GemStone environment is present

The mock suite is the main regression harness for layout persistence, window management, helper-window handoffs, debugger behavior, and Class Browser workflows.

Design Notes

Why the UI is window-heavy

The original tools are desktop/browser hybrids, not page-oriented CRUD screens. Preserving linked windows and arrows is important because the user's spatial layout carries meaning while exploring object graphs and class relationships.

Why so much Smalltalk is embedded in Python

GemStone data is cheapest to fetch in larger, encoded batches. For many views, one carefully shaped Smalltalk snippet is substantially faster and more robust than dozens of fine-grained bridge calls.

Why helper windows carry source-window IDs

Hierarchy, query, versions, and debugger windows are not independent views; they are assistants to a source browser or inspector. Persisting and restoring those relationships is what makes later actions like `Load Into Browser`, `Raise Related`, and `Close Group` behave consistently.